Waipapa
Taumata Rau
University
of Auckland

Lab 01

IDE, Version Control, GitHub Projects

27/07/2026

Brenda San Germán Bravo, Jack Haddad, Maoxiao (Mike) Ye, Zachary Saunders

Tēnā koutou katoa, Nau Mai, Haere Mai! Welcome!

Your tutors:

- Brenda San Germán
- Jack Haddad
- Maoxiao (Mike) Ye
- Zachary Saunders

Today's Focus

Prerequisites

- **Install Python**
- **Install PyCharm**
- **Apply for JetBrains Student Pack**
- **Create a GitHub account**

1. Set Up
   - Python
   - Version Control
   - IDE & Terminal
   - Virtual Environment
2. Hands-on Task 1
3. Getting Started with GitHub
4. GitHub Projects
5. Hands-on Task 2

Setting Up

Python

- High-level, general-purpose programming language
- Emphasises code readability
- Multi-paradigm, interpreted language.

Installing Python

Note: Python is already installed on lab computers.

1. Navigate to the Python download page
   - python.org/downloads

2. Choose the appropriate installer for your OS

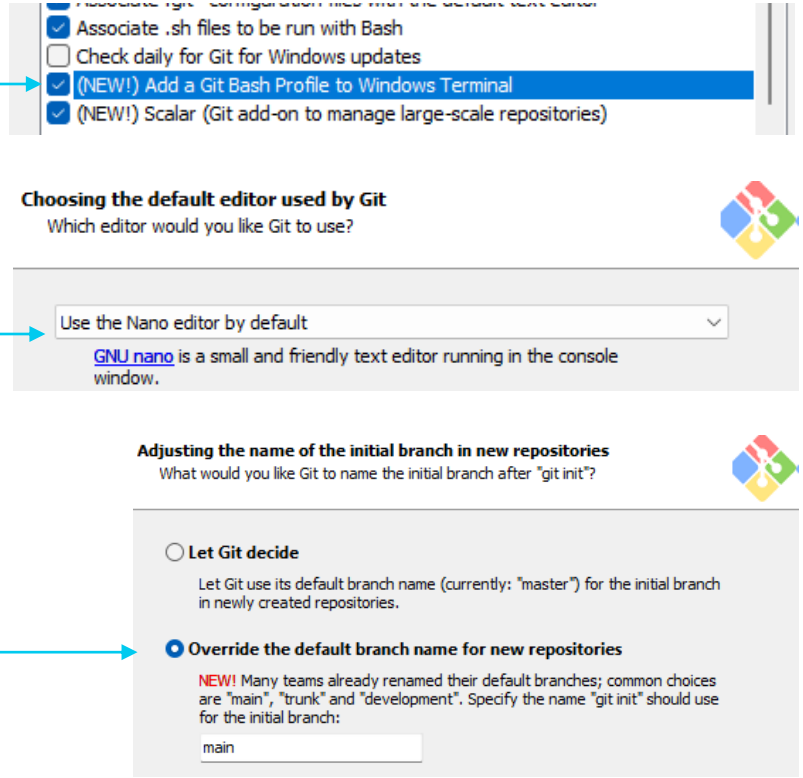
3. Run the installer and ensure you check the option **"Add python.exe to PATH"**

Git

- Open-source distributed version control system
- Almost exclusively used as version control in software development around the world
- Allows users to commit, branch, and merge.
- Supports secure commit signing

Installing Git

1. Navigate to the Git download page
   - git-scm.com/downloads/win
2. Choose the appropriate installer for your OS
3. Run the installer and use default options, you may choose to add **Git Bash** as a profile option in **Windows Terminal**
4. Choose a default editor, nano is the easiest
5. Override default branches to main
6. After this, choose all default values and install.



Note: Git is already installed on lab computers.

GitHub

- There are many providers that implement Git version control system as a service. Eg. Github, GitLab, Bitbucket, AWS CodeCommit, Gitea
- GitHub is the most widely used Git based version control service

Configuring Git to work with GitHub

- Step 1: Configure your Git username and email for every repository on your computer

  1. Open Git Bash (Any Terminal on macOS)
  2. Type the following command to set a Git Username
     - `git config --global user.name "Your Name"`
  3. Type the following command to set your email address
     - `git config --global user.email your_email@aucklanduni.ac.nz`

ProTip: To paste in Shell use your right click. (Ctrl+C is reserved for Cancel)

- Step 2: Authenticate GitHub to be used with the local machine
  - There are many ways you can authenticate your local Git client with GitHub
  - SSH is the most robust way to authenticate Git operations with GitHub.
    - Uses public–private key cryptography—no secrets are sent over the network.
    - Once your SSH key is linked, you never have to re-enter credentials for each push/pull.

Note: Lab computers reset nightly (wiping out your ~/.ssh folder)

Step 1: Generating a new SSH key

You need to generate a **Secure Shell Protocol** (SSH) key pair from your computer and link it to your GitHub account by adding public key to the GitHub account.

1. Open Git Bash.
2. Use the following command, replacing the email used in the example with your GitHub email address:

```
ssh-keygen -t ed25519 -C "email@aucklanduni.ac.nz"
```

3. When you're prompted to "Enter a file in which to save the key", you can **press Enter to accept the default file location.**
4. Enter a password (that you'll remember), or just **click enter for no password**

Pro Tip: macOS might need superuser(root) access. Start the commands with "sudo"

Step 2: Copy your public SSH key file

In step 1 you created a Key-pair consisting of a Private key file: 'id_rsa' and a Public key file: 'id_rsa.pub'

you need to locate and the contents of the id_ed25519.pub file to your clipboard

Via Terminal (Git Bash) is easy!

For **Windows**, in your Git Bash type:

```
clip < ~/.ssh/id_ed25519.pub
```

For **macOS**, in your terminal type:

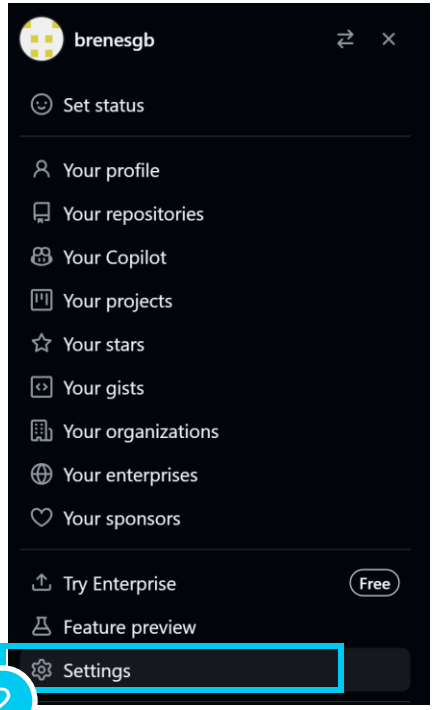
```
pbcopy < ~/.ssh/id_ed25519.pub
```

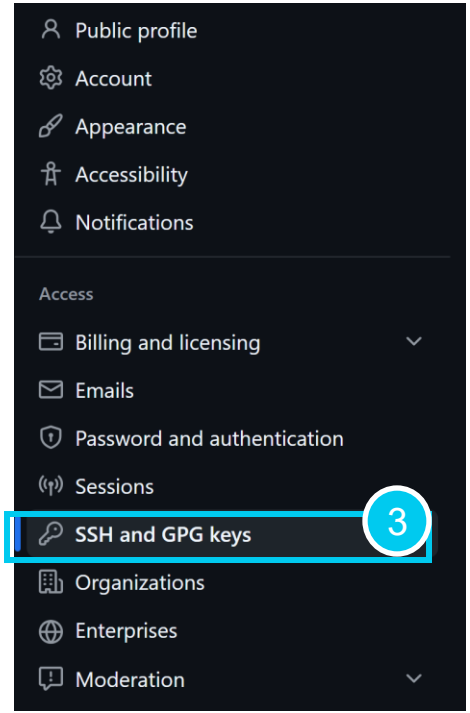
Note: If you find difficulty with the commands above, you can always locate the hidden .ssh folder, open the file in your favouiite text editor, and copy it to your clipboard.

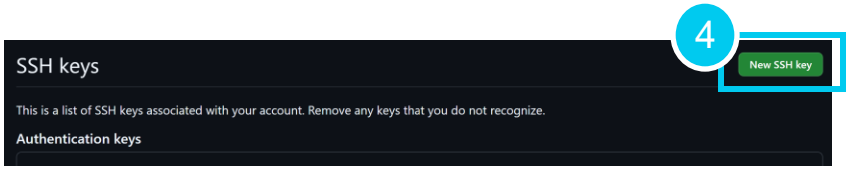
Step 3: Add the SSH key to your GitHub account

With your id_ed25519.pub freshly copied into your clipboard

1. Login into GitHub in your browser
2. In the upper-right corner, click your profile picture, then click Settings.
3. In the "Access" section of the sidebar, click SSH and GPG keys.
4. Click New SSH key or Add SSH key.
5. In the "Title" field, add a descriptive label for the new key.
6. Select the type of key as "Authentication"
7. In the "Key" field, paste your public key.
8. Click Add SSH key.


brenesgb
Set status
Your profile
Your repositories
Your Copilot
Your projects
Your stars
Your gists
Your organizations
Your enterprises
Your sponsors
Try Enterprise
Free
Feature preview
Settings
2


Public profile
Account
Appearance
Accessibility
Notifications
Access
Billing and licensing
Emails
Password and authentication
Sessions
SSH and GPG keys
3
Organizations
Enterprises
Moderation


SSH keys
4
New SSH key
This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.
Authentication keys

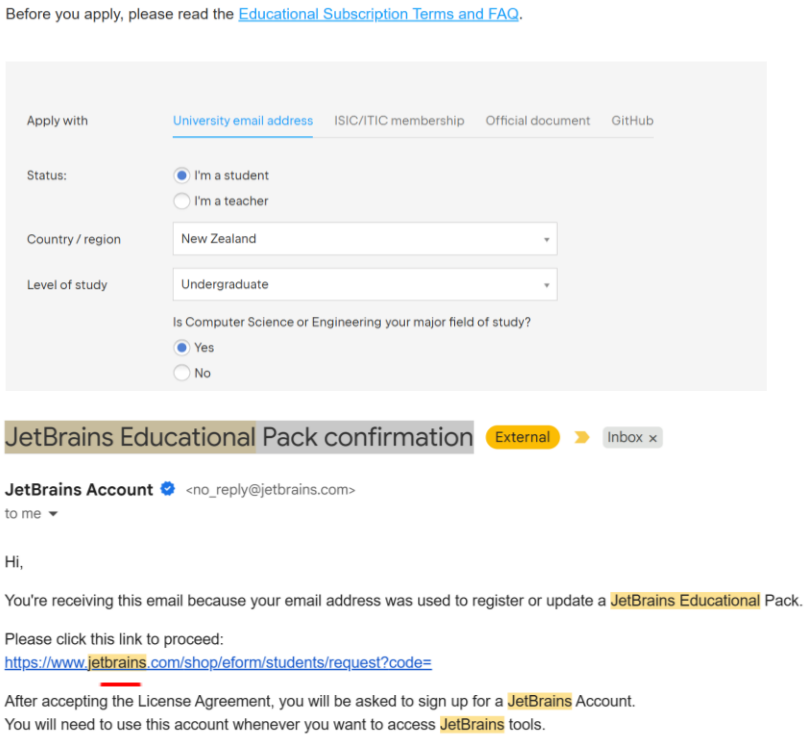

Add new SSH Key
Title
Personal laptop
5
Key type
Authentication Key
6
Key
Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'
Ctrl + C
7
Add SSH key
8

PyCharm

- Full-featured Integrated Development Environment (IDE) for Python
- Familiar UI if you are a Java/Android developer
- Easier to manage a project and a virtual environment

Free JetBrains Student Pack

1. Navigate to jetbrains.com/shop/eform/students
2. Choose the option Apply with **University email address**
3. Fill in the form with your information and apply.
4. Wait for a JetBrains Educational Pack **confirmation email** and follow the instructions

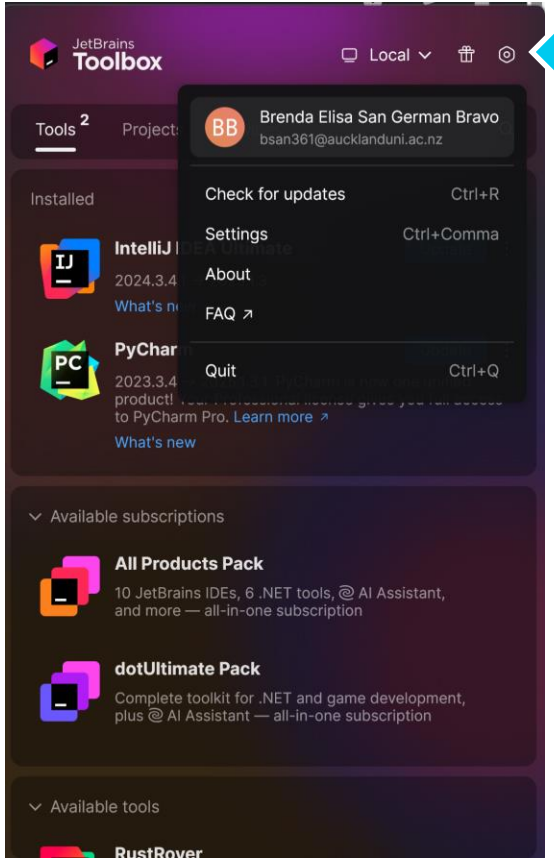


Installing PyCharm

1. Download [JetBrains Toolbox](jetbrains.com/toolbox-app)
   - jetbrains.com/toolbox-app

2. In Toolbox, in the top right, log in to your JetBrains account
   - Login with Google using your university email

3. Once logged in, Download PyCharm



IDE, Virtual Environment and Terminal

Virtual Environment: What & Why

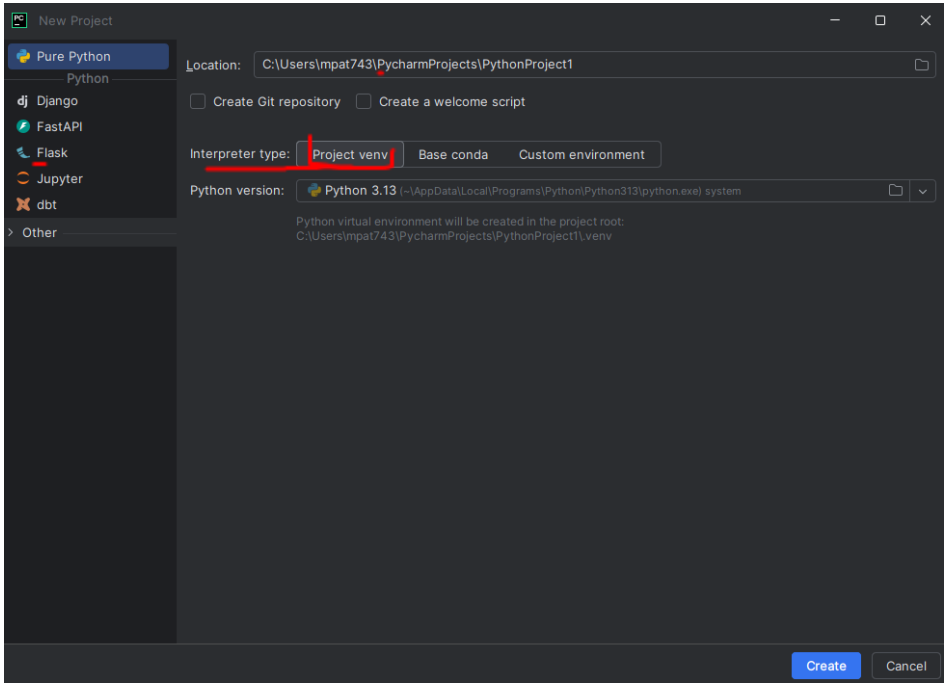
- **What**: as the name suggests, is a virtual isolation of Python version and its installed packages.
- **Why**: Imagine you're doing a project this year. You will probably install few packages to support your project. Let's say you install Numpy package. Its current version is 1.25.1. So you install that. After few years, you are going to do a project again that uses Numpy. But that library is now updated to 2.10.2. If you install that in your laptop, you cannot run your old project as it uses 1.25.1. This is why having a virtual environment dedicated to each project is really important.

Note: A Python **package** is a bundle of reusable code that you can install via pip—most come from the Python Package Index (PyPI) so you don't have to rewrite common functionality.

New PyCharm Project

PyCharm will automatically create a venv for you!

1. Create a Pure Python project in the location you want.
2. Select Project venv as the Interpreter type
3. Select the Python version from the dropdown, if you want, you can also install the latest git here.
4. Click Create!



Virtual Environment: save package info

- Ensure another person can setup your project without a hassle by providing them information on which versions of which libraries you have used in the project.
- It's only for packages, not for Python version.
- Make sure you have the matching major and minor version for Python (Patch version usually doesn't matter i.e. 3.13.X the X is the patch).

```
# Ensure your virtualenv is active before running these commands

# Export a snapshot of all installed packages (with exact versions) to "requirements.txt"
pip freeze > requirements.txt

# To recreate the exact same set of packages on another machine or project
# Install from requirements.txt
pip install -r requirements.txt
```

Terminal

To become a better developer, you should get familiar with terminal Unix commands as most servers do NOT have a graphic UI.

On Windows

- Press ”Windows” key and look for ”PowerShell”

On Mac

- Type terminal or cmd

Note: Windows was not built on Unix.

If you see errors like `mv: command not found`, you can either

- install Git Bash, or
- enable the Windows Subsystem for Linux (WSL), or
- use PowerShell equivalent command

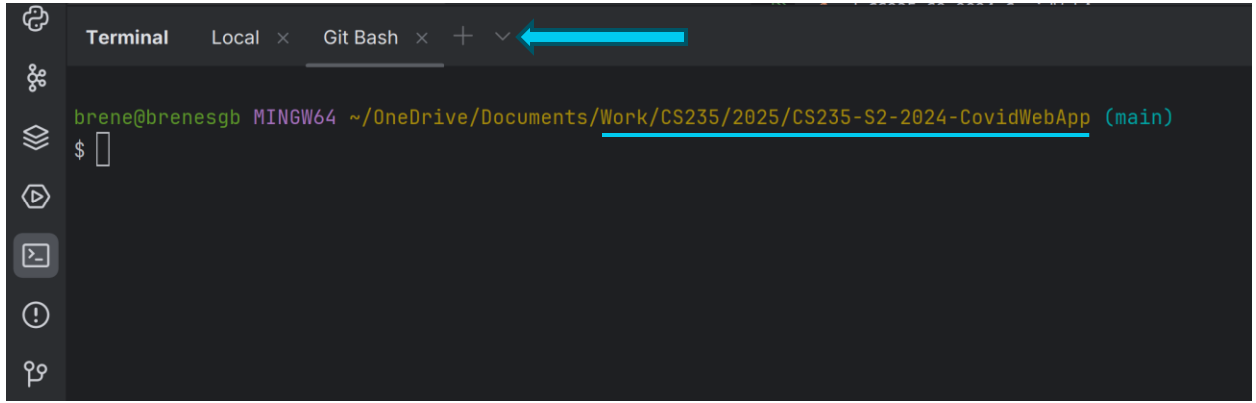
```
# Navigate to your "home" directory
cd ~

# Create a folder called "workspace"
mkdir workspace

# Rename "workspace" to "my_projects"
# The same command is also for MoVing files
mv workspace my_projects

# ReMove a folder; "-r" for Recursive
rm -r my_projects
```

Terminal - PyCharm

Luckily PyCharm has an in-built terminal window.

1. In the bottom-left, click the terminal icon
2. In the terminal panel, click the down arrow ⌄ next to the +
3. Select Git Bash and run the commands to the right.

Note: Notice how the terminal working directory is the current project root! No need to cd manually!



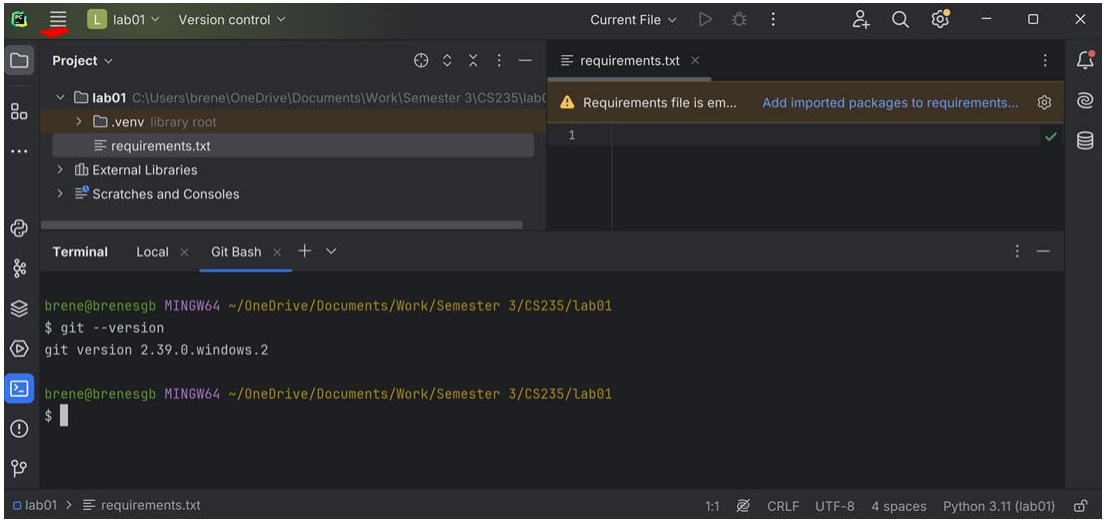
Task 1

Setting Up your Environment

Task 1 – Setting Up the Environment

Show the tutor your PyCharm editor with the following:

- A project with
  - .venv
  - requirements.txt file
- Open terminal (set to GitBash) with the output from
  - `git --version`



(You've got this! If you followed the previous slides, you'll breeze through these checks.)

Git & GitHub

Setting up GitHub

There are different tools you can choose to work with Git and GitHub

GitBash (CLI) 

- Use standard Unix
  - navigate filesystem
  - automate tasks
- Useful in most remote or production machines.

GitHub Desktop (GUI) 

- Visual tool for commits, branches, merges
- Simplifies authentication and remote setup

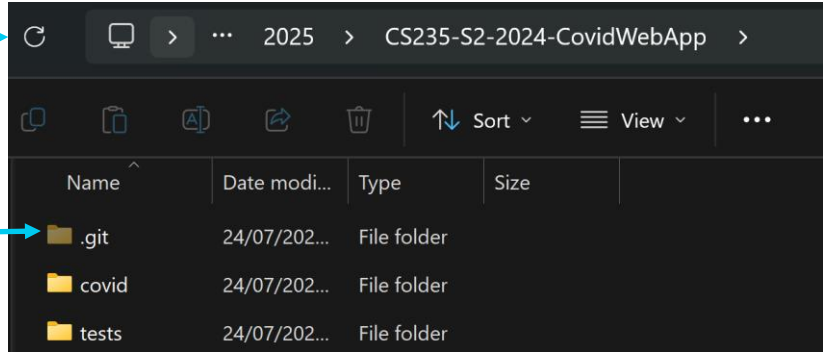
IDE Integration 

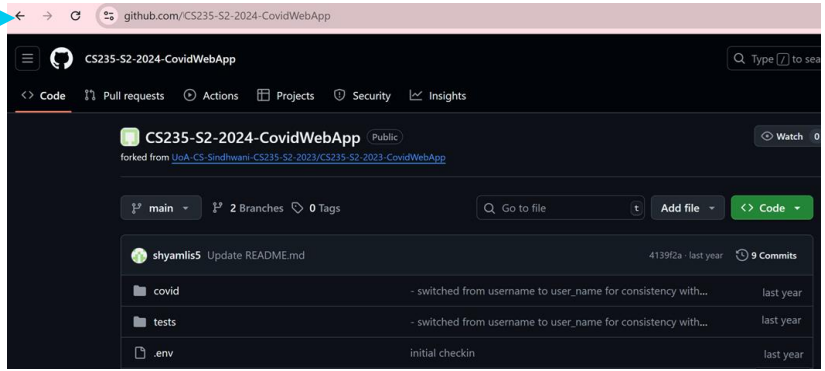
- Built-in VCS panel
- Streamlines coding + version control in one application

You can mix and match any of these – pick the setup that feels most comfortable and switch later if you want!

Git basics

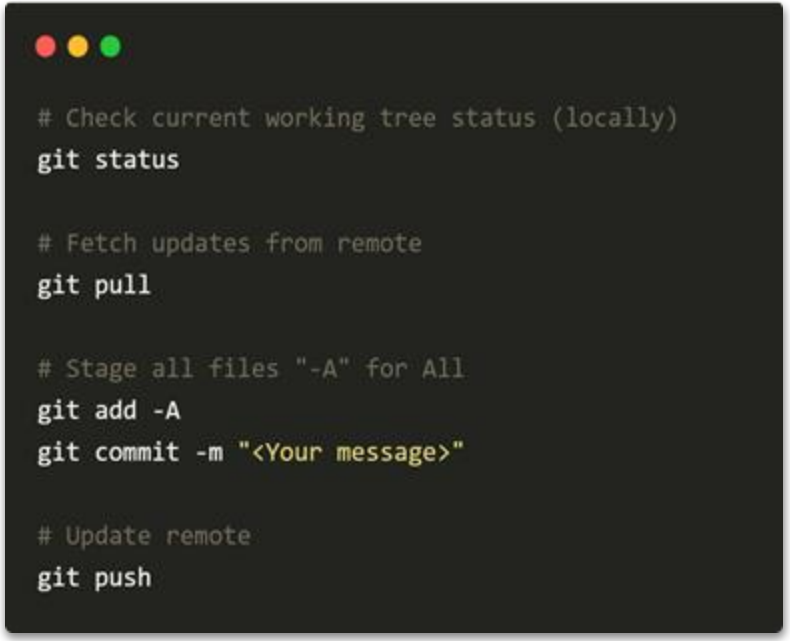
- Working Copy (Your PC)
  - Edit files in your project folder
  - Git watches everything you change here
- Local Repo (.git folder)
  - Hidden folder inside your project
  - Stores commits and metadata.
  - Managed by **git add** + **git commit**
  - Nothing has gone to the internet yet.
- Remote Repo (origin)
  - Hosted on a server somewhere (GitHub/GitLab/etc.)
  - Central place to share & collaborate
- Syncing
  - **git push**: upload your local commits up to “origin” so others can see them.
  - **git pull**: fetch new commits from “origin” and merge them into your working copy.





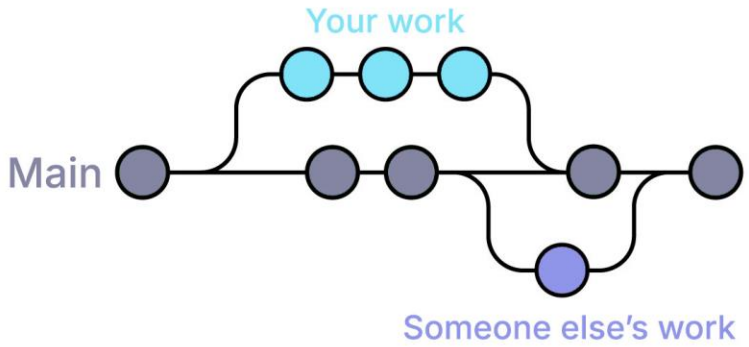
Git basic commands

- **`clone`** - Copies a remote repos into your current directory.
- **`init`** - Creates a new empty repo in your current directory.
- **`add`** - Marks changes in your working folder so they'll be included in your next commit (you're "staging" them).
- **`status`** - Lists changes in working directory, and staged files.
- **`commit`** - Takes all staged changes and records them as a snapshot in your local history, along with your message describing what you did.
- **`pull`** - Incorporates changes from 'origin' into local repo.
- **`push`** - Incorporates changes from local repo into origin.



Source: https://docs.nesi.org.nz/Getting_Started/Cheat_Sheets/Git-Reference_Sheet

Branching and Merging



A Branch is a separate line of development where you can work on new features or fixes.

Changes you make on a branch won't affect other branches until you merge them back

- **`git branch <name>`**: makes a copy of your current code state under a new name.
- **`git checkout <name>`**: switch into that branch.
- **`git merge <branch>`**: brings changes from one branch into the branch you're on, combining work.

A **merge conflict** happens when the same lines were changed in both branches, Git asks you to choose which version to keep.

You will need to edit the file between the markers added by Git to pick or combine versions.

```
Automatic merge failed; fix conflicts and then commit the result.
```

```
<<<<<<< HEAD
Your change here
=======
Incoming branch change here
>>>>>>> feature
```

Gitignore

- Make sure `.gitignore` is in the root directory
- Here is a collection of templates from official GitHub
- Git cannot track binary files. You should consider to add them to gitignore.

e.g. your code generates plots as PNG files; each time you run the code; Git will think you changed all plots.

- Some examples you should consider to ignore: .PDF, .JPG, .PNG, .log, .lib

Best practice

- **Test** first, commit second
- **Don’t commit unfinished work** to main
- Write meaningful commit messages but make it short.

  E.g., “~~updated main.py~~” Bad; “Add retrain function to Model class” Good!
- More info on How to Write a Git Commit Message
- Commit small and often. E.g., Don’t commit 100 files at once. Each job is one commit
- Enforce standards for the project. E.g., Have a single formatting convention, use a standard naming convention

https://cbea.ms/git-commit/

GitHub Projects

GitHub Projects

- It is another tool from GitHub for planning and tracking your work.
- It helps organise your work into a simple workflow so your team sees the big picture*.
- You can visualise owners, priorities, due dates, and progress in one place.
- Projects are built from your issues/PRs and stay in sync both ways.
- You can switch between views, depending on which perspective you need (Board or Timeline/Roadmap.
- You can build simple progress charts from project data to share with your team.

**Look out for Week 10 to deepen this with Agile: Scrum, XP, Kanban.*

Task 2

Getting Started with GitHub

You will need to work in pairs

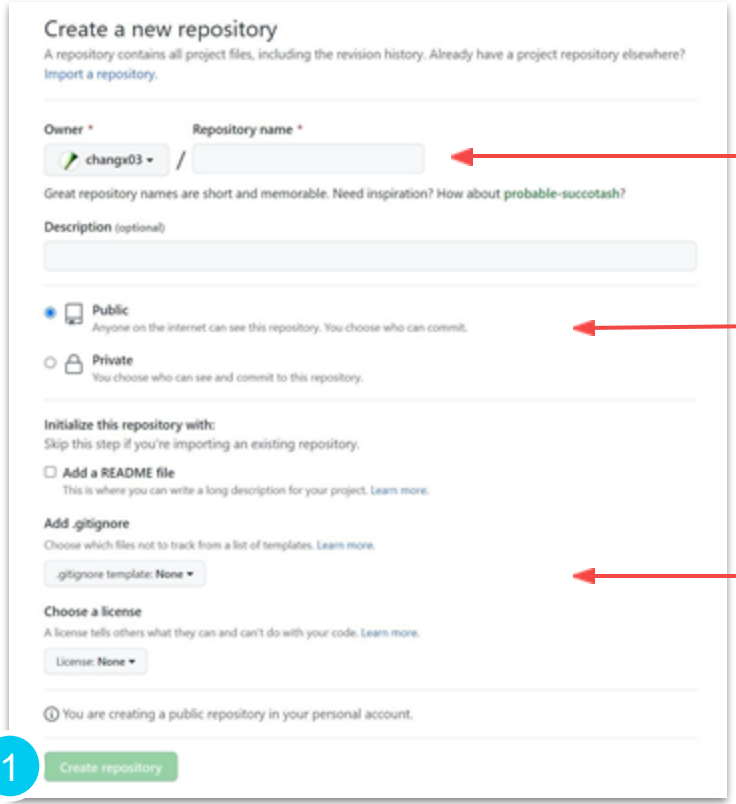
Show the tutor your final repository with commits of both team members

Team Member A

Task 2: Team member A

1. Create a new remote repository in GitHub
2. Convert your existing project into a local Git repository. Run:
   ```
   git init
   ```
3. Download the GroupMembers.html file from Canvas > Assignments > Lab1 under Hands-on tasks and add it to the root folder of project created in Task 1
4. Stage your changes
   ```
   git add .
   ```
5. Make your first commit
   ```
   git commit -m "First commit"
   ```
6. Synchronise the local repository with the remote in GitHub, make sure you copy the SSH url from your remote repo
   ```
   git remote add origin REMOTE-URL
   ```
7. Create the main branch:
   ```
   git branch -M main
   ```
8. Make your first push. Run:
   ```
   git push -u origin main
   ```
9. Share your repo with Team member B

Creating a new repository on GitHub

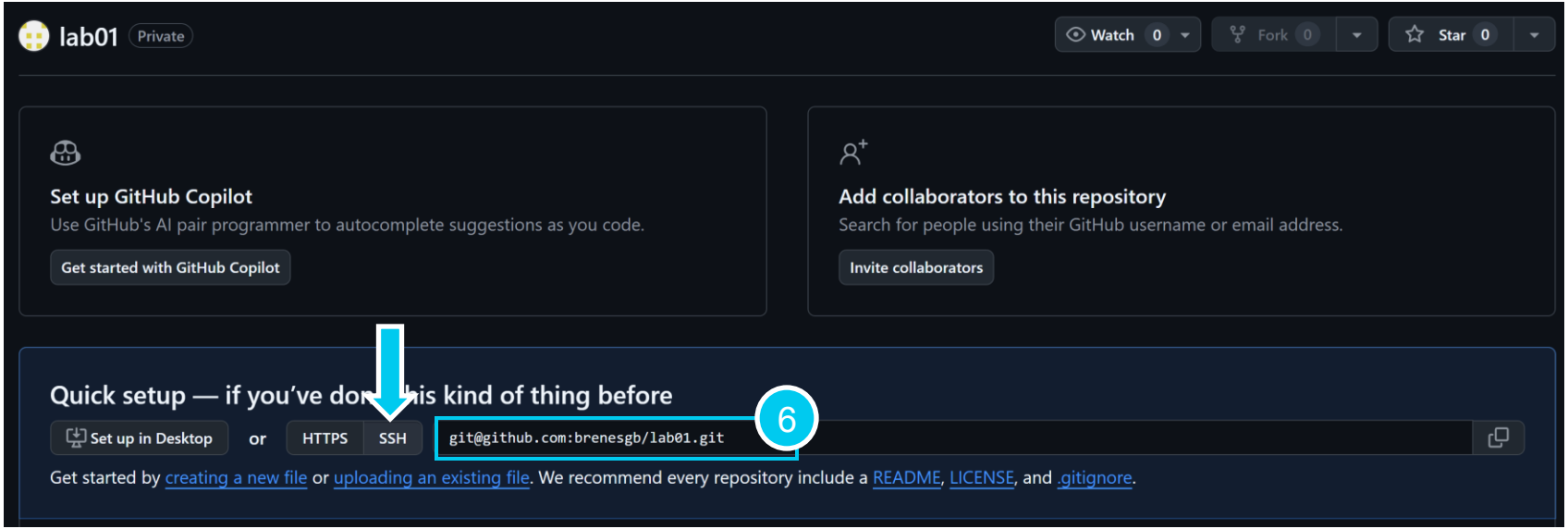


Repo name must be unique

Do you want this repo to be public?

Which files will be ignored?

Find the SSH url from your remote repo



Create local Git repo and synchronise with remote in GitHub

2
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01
$ git init
Initialized empty Git repository in C:/Users/brene/OneDrive/Documents/Work/CS235/2025/lab01/.git/
```

4
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (master)
$ git add .
warning: in the working copy of '.idea/inspectionProfiles/Project_Default.xml', LF will be replaced b
warning: in the working copy of '.idea/inspectionProfiles/profiles_settings.xml', LF will be replaced
```

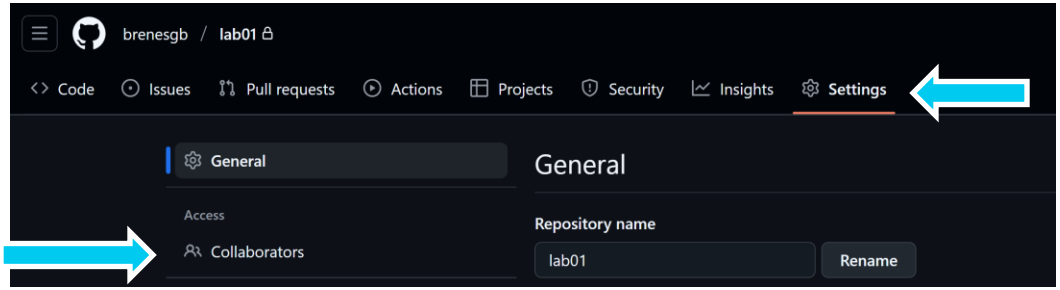
5
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (master)
$ git commit -m "First commit"
[master (root-commit) 5d195c6] First commit
 9 files changed, 90 insertions(+)
 create mode 100644 GroupMembers.html
 create mode 100644 requirements.txt
```

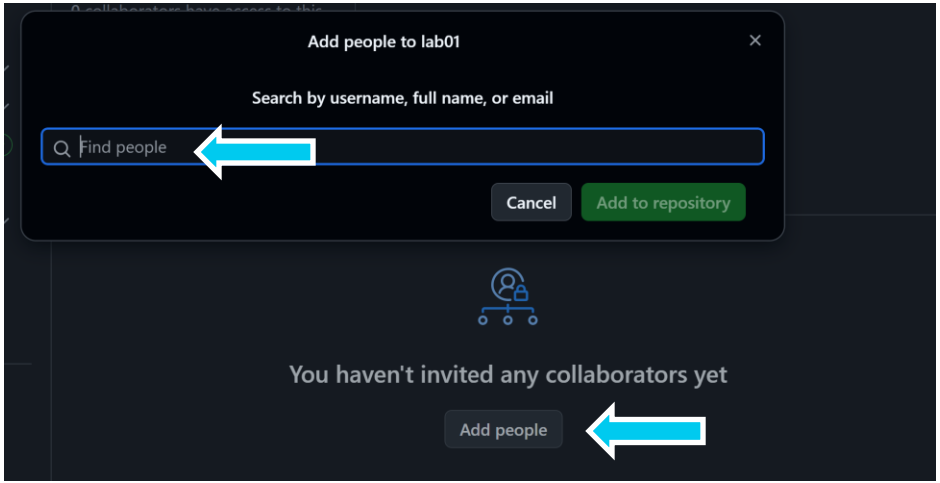
6
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (master)
$ git remote add origin git@github.com:brenesgb/lab01.git
```

7
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (master)
$ git branch -M main
```

8
```
brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (main)
$ git push -u origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 20 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (13/13), 2.12 KiB | 724.00 KiB/s, done.
```

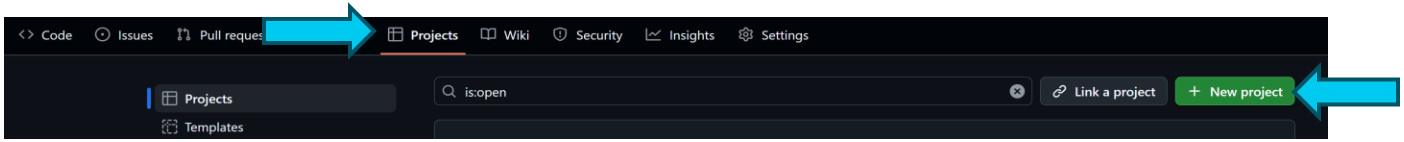
Share remote repository

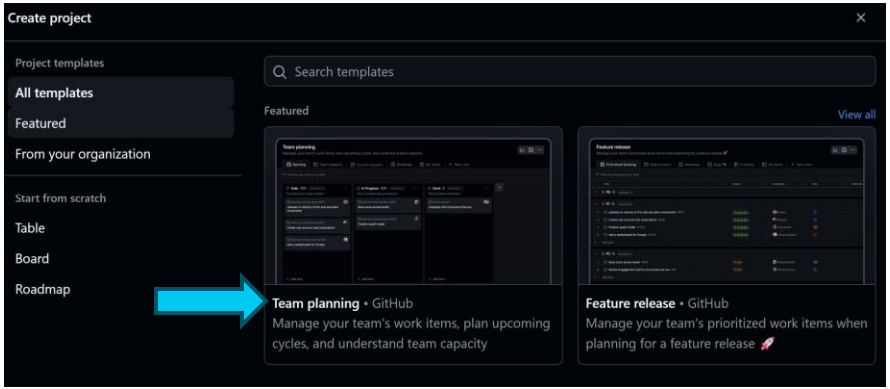


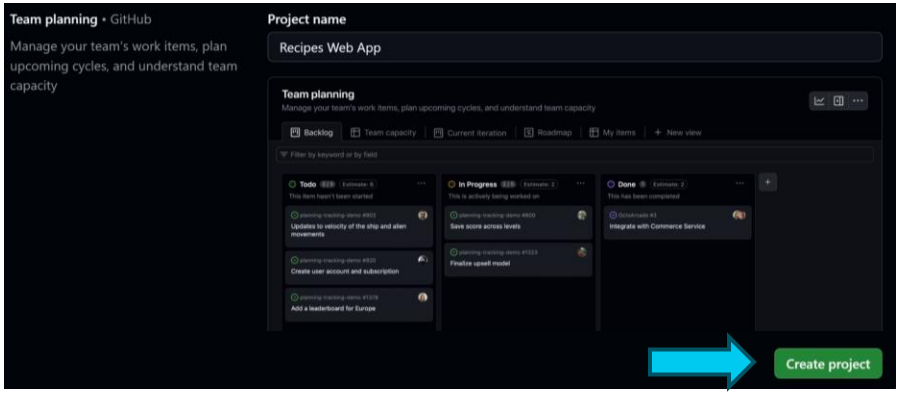


Set up a Project

1. Set up your GitHub project and add team member B as a collaborator





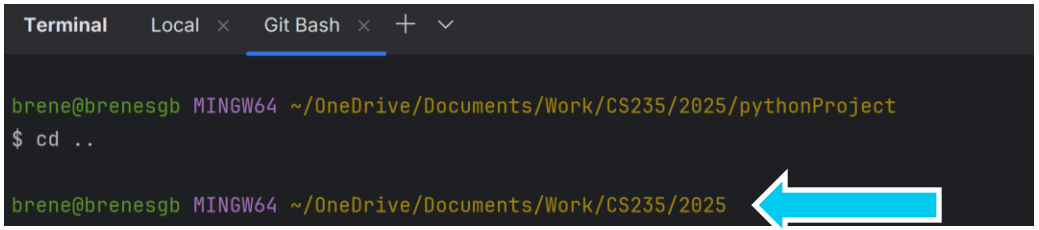


Team Member B

Task 2: Team member B

1. Open Git Bash

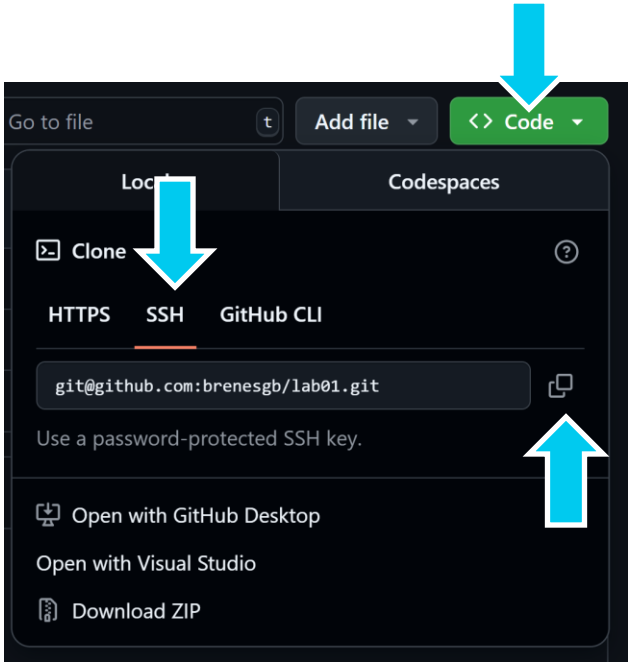
2. Use cd to navigate to where you want your project to be saved (preferably a different location than in Task 1)



3. Clone the repository from your teammate's GitHub

```
git clone <ssh remote-url>
```

Clone – Download a repository



- Copy the URL from the remote repo you want to clone
- Make sure you copy the URL under SSH
- In your Git Bash terminal run:

```
git clone <ssh remote-url>
```

Task 2

2. Populate your backlog on the Project created by Member A:
    - Create new issues (one small task per card of things you will need to do).
    - Both team members add cards from the Issues you created
    - Place the cards depending on where they are on the workflow (To Do, Doing, Done).
    - Assign an owner, set priority and due date for each card.

(1 Mark) - Show tutors a Project board set up for your initial tasks for this course.

Task 2:
Make changes and push

1. Add the team members' names (replace "Team member one")
2. Stage your changes
   `git add .`
3. Make your first commit
   `git commit -m "Added team names"`
4. Make your first push
   `git push -u origin main`

```
<div style="text-align: center;">
  <h1>My Group</h1>
  <br>
  <h3>Group member list:</h3>
  <!-- Please put team members here -->
  <div>
      <p> Brenda San German </p>
      <p> Jack Haddad</p>
  </div>
</div>
```

```
Terminal   Local ×   Git Bash ×

brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   GroupMembers.html

no changes added to commit (use "git add" and/or "git commit -a")

brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (main)
$ git add .

brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (main)
$ git commit -m "added team member's names"
[main e8404c8] added team member's names
 1 file changed, 1 insertion(+), 2 deletions(-)

brene@brenesgb MINGW64 ~/OneDrive/Documents/Work/CS235/2025/lab01 (main)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 362 bytes | 362.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:brenesgb/lab01.git
   466c281..e8404c8  main -> main
```

Thank You!

See you next week!